DAVINCI: Decentralized autonomous vote integrity network with cryptographic inference

Pau Escrich[1], Marta Bellés[2], Jordi Piñana[1], Lucas Menéndez[1], and Jose Luis Muñoz-Tapia[3]

[1] Vocdoni Association – {pau,painan,lucas}@vocdoni.org
[2] Institution – email@email.com
[3] Polytechnic University of Catalonia – jose.luis.munoz@upc.edu

Last update: August 25, 2025

Abstract. DAVINCI is the evolution of the Vocdoni voting protocol, designed to empower civil society by providing essential tools for secure, verifiable, and anonymous digital voting. Leveraging recent advancements in zero-knowledge (ZK) proof systems and blockchain technology, DAVINCI transitions to a specialized ZK-rollup system that inherits network security from settlement layers like Ethereum mainnet. The system relies on cryptographic proofs to ensure integrity and security, eliminating the need for centralized authorities. By integrating ZK-SNARKs and threshold homomorphic encryption, DAVINCI enables end-to-end verifiability, privacy, and trustlessness in the voting process. The protocol employs a distributed key generation among sequencers, coordinated via Ethereum smart contracts, and uses Ethereum data blobs for data availability. With a focus on accessibility, scalability, receipt-freeness, and automation, DAVINCI aims to facilitate high-frequency and low-cost voting, fostering mass adoption of e-voting and simplifying civil participation. Moreover, the introduction of the Vocdoni token aligns incentives among participants, ensuring the system's sustainability and enabling decentralized governance. Finally, the design of DAVINCI is grounded in practicality; all components are implemented using current technologies and have undergone proof-of-concept testing. In sum, the proposed architecture is not merely theoretical but a viable solution ready for short-term deployment.

Pieces of text marked in **blue** are heavily work in progress. Additionally, there are these general tasks left:

- Consider separating the roles of the sequencers from key generators.
- Change (C_1, C_2) to `ciphertext` (in protocol flow and figures).
- Notation: are voteIDs called Nullifiers?
- Consider the scenario with deterministic signature (no Eth compatible) that can be used as nullifier.
- Do new circuit1 + circuit2.
- Do last results circuit.

Table of Contents

1 Introduction

Online voting remains one of the most studied yet elusive applications in applied cryptography. As digital services expand across both public and private sectors, secure and universally verifiable online voting systems have become an increasingly desirable goal. Remote electronic voting promises scalability, accessibility, and administrative efficiency; yet the design of systems that simultaneously ensure privacy, integrity, coercion resistance, and transparency under realistic adversarial models remains a formidable challenge. Numerous deployments have revealed deep-rooted usability flaws and security gaps, particularly when verifiability mechanisms are either poorly understood by users or reliant on centralized infrastructure.

Against this backdrop, the Vocdoni project was initiated in 2018 with the aim of rethinking online voting from first principles. The name Voĉdoni, meaning "to give voice" in Esperanto, reflects the project's foundational goal: to empower collectives—from small associations to millions of citizens—to engage in secure and verifiable decision-making, regardless of technological or institutional barriers. Central to this vision was the idea that voting is not limited to formal governmental elections but serves as a more general-purpose mechanism for collective signaling. Vocdoni introduced a fully anonymous, end-to-end verifiable voting system designed to operate efficiently across a range of devices, including smartphones. To support these goals, the team deployed a custom infrastructure emphasizing resilience, neutrality, and transparency.

Technically, the architecture of Vocdoni was grounded on a bespoke Byzantine fault tolerant (BFT) layer-1 blockchain, named Vochain. At the time, zero-knowledge (ZK) proof systems such as ZK-SNARKs were emerging but not yet practical for most deployments. Vochain provided a performant and low-cost environment (achieving approximately 700 transactions per second) where advanced cryptographic tools could be used without the constraints imposed by blockchains based on the Ethereum virtual machine (EVM). The ability to issue voting transactions without requiring user fees enabled broader accessibility. Over several years of development and deployment, this architecture proved both viable and valuable in practice. However, broader adoption as a universal voting protocol highlighted the need for further architectural refinements and stronger formal guarantees.

In this work, we introduce DAVINCI, a new protocol that builds upon the lessons and conceptual groundwork laid by Vocdoni. DAVINCI adopts a modular design and integrates state-of-the-art cryptographic tools, including succinct ZK proofs, improved bulletin board constructions, and robust coercion resistance mechanisms—reflecting the most recent advances in academic research. Unlike monolithic designs, DAVINCI is conceived as a composable primitive: a foundational layer intended to support secure and verifiable voting in diverse contexts, from blockchain governance to institutional elections. This shift in design philosophy aims to address long-standing challenges in the field, offering a cleaner abstraction with clearer security boundaries and formal underpinnings.

1.1 Contributions

This work introduces DAVINCI, a modular and verifiable protocol for digital voting, designed to support privacy-preserving, auditable, and adaptable election processes across diverse governance contexts. The protocol models voting as a constrained state machine, in which each valid operation is expressed as a formally defined state transition, enforced through succinct ZK proofs. That is, at the core of the system lies a set of reusable arithmetic circuits that implement voter authentication, encrypted ballot validation, state transitions, and tally finalization. These circuits are optimized for off-chain proving and are compatible with modern ZK-SNARK systems. Application-layer state is maintained off-chain and committed on-chain using cryptographic accumulators, specifically Merkle trees. The protocol defines two such trees: one for the voter registry (the census) and one for the voting state. These trees enable voters and verifiers to produce efficient inclusion and transition proofs, while minimizing on-chain storage and computation.

To coordinate protocol execution, DAVINCI includes a set of Ethereum smart contracts that mediate the setup of elections, manage finalization procedures, and verify submitted proofs of correctness. A standardized transaction and data encoding format supports all stages of the process, including vote casting, aggregation, and result publication. This approach ensures that protocol interactions remain deterministic and interoperable across clients and backends. The system also introduces the Vocdoni token, which is used to align incentives between participants. Altogether, the design aims to balance auditability and privacy

while reducing trust assumptions on infrastructure providers. By cleanly separating concerns and providing modular interfaces, the protocol is intended to serve as a foundation for both practical deployments and future extensions, such as integration with alternative identity systems, off-chain data availability layers, or other consensus backends.

- The Vocdoni voting protocol.
- The Vocdoni ballot protocol.

1.2 Related work

- State of the art of e-voting: `https://azkr.org/blog/evoting-review/`.

1.3 Paper organization

- Section 2: background.
- Section 3: high overview of the election process. We omit technical details.
- Section 4: cryptographic primitives used in the voting process. A description of the primitives but also the specific protocols being used and the details of their instantiation.
- Section 5: description of the voting process.
- Section 6: a proposed protocol for ballots. The rules of this protocol are implemented in the process management smart contracts.
- Section 7: the Vocdoni token.
- Section 8: analysis of the proposed protocols. It includes a security discussion of the properties of the protocol, implementation details, and a performance evaluation.
- Section 9: we finish with some last remarks.

2 Background

2.1 Interplanetary file system

- Brief introduction to IPFS.

2.2 Ethereum

- Ethereum blockchain (say transactions have a fee cost).
- Ethereum smart contracts.
  Ethereum smart contracts are employed to orchestrate the voting process, manage state transitions, and store critical data, such as the current state root and encryption public keys. These smart contracts provide a trustless environment, ensuring that all participants can be confident that the voting process rules are correctly enforced.
- Ethereum data blobs.
  EIP-4844 is used for data availability, enabling the storage of state transition data in the form of Ethereum data blobs. This mechanism allows sequencers to collaborate on the construction and verification of the voting process state, ensuring efficient and decentralized data availability.

2.3 Zero-knowledge proofs

- Introduce ZK-SNARKs.
- Introduce arithmetic circuits (arithmetic circuit satisfiability).

3 Protocol intuition

All the voting process is governed by the rules encoded in three smart contracts: the *process management*, the *sequencer registry*, and the *results verification* smart contract. All these three contracts are deployed on the Ethereum blockchain, and ensure that the execution of all steps is transparent and unalterable. The protocol consists of 5 phases, as illustrated in Figure 1, and described below.

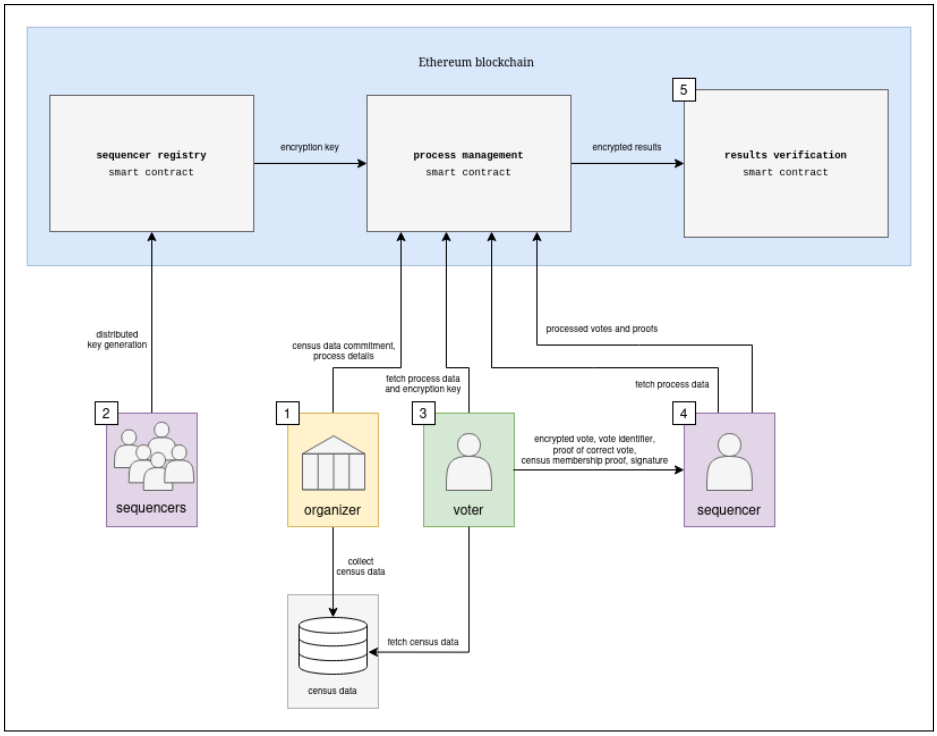

Fig. 1. Overview of the protocol flow.

1 *Voting setup.* In the first phase, the organizing entity gathers all census data and generates a cryptographic commitment to this data. Additionally, the organizer defines the voting parameters, including the number of voting options and the vote-counting mechanism (e.g., weighted voting, quadratic voting, etc.). Once these details are set, the organizer submits a transaction to the process management smart contract on the Ethereum blockchain. This transaction publicly records the voting parameters and census commitment, ensuring transparency and preventing any subsequent alterations to the voting setup.

2 *Encryption key generation.* A designated group of participants, referred to as *sequencers*, collaboratively generate a shared encryption public key, which voters will use to encrypt their votes. This key is established through a distributed key generation (DKG) protocol, ensuring that no single party can control or reconstruct the corresponding private key independently. Sequencers have a dual role in the protocol. In addition to generating the encryption key, they are also responsible for collecting votes from voters during the election. Consequently, their public key and the necessary information to contact them off-chain must be registered in the sequencer registry smart contract. Once the encryption public key is securely computed and published in the sequencer registry smart contract, the voting phase can start.

3 *Voting.* Voters select their preferred option according to the voting rules established by the organizer and captured in the process management smart contract. Instead of submitting their votes directly on-chain, they send them off-chain to a sequencer of their choice for processing. To ensure privacy, votes are encrypted

using the available encryption public key. Additionally, users compute a unique vote identifier that will allow them to verify that their vote has been included in the final result. Alongside the encrypted vote and the vote identifier, each voter must also provide the following data: a proof of valid voting, demonstrating that the vote complies with the election rules; a proof of eligibility, verifying that they are registered in the census; and a proof of identity ownership, in the form of a digital signature, confirming that they are the legitimate voter and are not impersonating someone else. Finally, to mitigate coercion and vote-buying, the protocol allows voters to overwrite their vote any number of times during the voting phase.

4 *Votes collection.* During this phase, sequencers collect encrypted votes from multiple voters along with their corresponding proofs, and verify the validity of these submissions. That is, sequencers verify the signatures, to ensure the votes were cast by legitimate voters, they verify the proof of compliance, confirming that each vote adheres to the polling rules, and the census membership proofs against the commitment to the census data that was originally registered in the process management smart contract. Once the sequencers have processed and verified all votes, they must prove that these verifications were performed correctly. Instead of submitting individual verifications for each vote, they generate a single ZK proof that attests to the correctness of all verifications. Additionally, the sequencers reencrypt the votes and generate a proof of the correct reencryption computation. While this process does not alter the final tally, it prevents voters from decrypting their original vote. This step mitigates vote selling or coercion, as voters are no longer able to prove their choice to third parties. Finally, the sequencers submit the reencrypted votes along with their verification and reencryption proofs to the process management smart contract. The smart contract verifies all the proofs provided by the sequencers to check that they complied with the protocol.

5 *Results verification.* At the conclusion of the voting period, the smart contract ceases to accept new state updates, effectively finalizing the process. The final state is then available on-chain for verification, providing an immutable record of the voting outcome.

4 Cryptographic primitives

In this section, we detail the cryptographic primitives used in Section 5. We briefly introduce elliptic curves, hash functions, Merkle trees, encryption schemes, and proof systems used in the protocol. We also specify at each step the concrete parameters with which each of the primitives is instantiated.

4.1 Elliptic curves

DAVINCI relies on multiple elliptic curves to ensure interoperability with Ethereum, compatibility with available cryptographic primitives, and efficient in-circuit operations. On the one hand, SECP256K1 [7] is used because Ethereum public keys are elements of this curve. As DAVINCI assumes each voter holds a standard Ethereum address, cryptographic operations such as digital signatures and identity verification rely on SECP256K1 to match the Ethereum ecosystem. On the other hand, BN254 [12] is chosen because it is the curve supported by Ethereum's precompiled contracts for ZK proof verification, which makes it ideal for verifying SNARK proofs on-chain with minimal gas cost. Then, BabyJubjub [3] is used as an inner curve for elliptic curve operations within arithmetic circuits. Finally, BLS12-377 [6] and BW6-761 [8] are used to enable recursive proof composition. More specifically, BLS12-377 acts as the inner curve for constructing proofs, while BW6-761 serves as the outer curve that verifies those proofs within larger ZK-SNARK circuits. This pairing enables succinct verification and composability of ZK-SNARKs within other ZK-SNARKs, which we use for DAVINCI 's aggregation and state transition logic. Below, we give the details of these curves.

Parameters.

- $p =$ `0xfffffffffffffffffffffffffffffffffffffffffffffffffffffffefffffc2f` (256-bit prime).
- $q =$ `0xfffffffffffffffffffffffffffffffebaaedce6af48a03bbfd25e8cd0364141` (256-bit prime).
- $r =$ `0x2523648240000001ba344d80000000086121000000000013a700000000000013` (254-bit prime).
- $s =$ `0x2523648240000001ba344d8000000007ff9f800000000010a10000000000000d` (254-bit prime).
- $t =$ `0x60c89ce5c263405370a08b6d0302b0bab3eedb83920ee0a677297dc392126f1` (251-bit prime).

- $u =$ 0x122e824fb83ce0ad187c94004faff3eb926186a81d14688528275ef8087be41707ba638e584e9190 3cebaff25b423048689c8ed12f9fd9071dcd3dc73ebff2e98a116c25667a8f8160cf8aeeaf0a437e69 13e6870000082f49d00000000008b (761-bit prime).
- $v =$ 0x01ae3a4617c510eac63b05c06ca1493b1a22d9f300f5138f1ef3622fba094800170b5d4430000000 8508c00000000001 (377-bit prime).
- $w =$ 0x12ab655e9a2ca55660b44d1e5c37b00159aa76fed00000010a11800000000001 (253-bit prime).

Elliptic curve groups.

- SECP256K1 curve: $E^{\mathsf{SEC}}/\mathbb{F}_p$ defined by equation $E^{\mathsf{SEC}} : y^2 = x^3 + 7$, with group $\mathbb{G}^{\mathsf{SEC}}$ of prime order q.
- BN254 curve: $E^{\mathsf{BN}}/\mathbb{F}_r$ defined by equation $E^{\mathsf{BN}} : y^2 = x^3 + 3$, with subgroups $\mathbb{G}_1^{\mathsf{BN}}, \mathbb{G}_2^{\mathsf{BN}}$ of prime order s, and an efficiently computable pairing $e : \mathbb{G}_1^{\mathsf{BN}} \times \mathbb{G}_2^{\mathsf{BN}} \to \mathbb{G}_T^{\mathsf{BN}}$, where $\mathbb{G}_T^{\mathsf{BN}} \subset \mathbb{F}_{s^{12}}$ and has order s.
- BabyJubjub curve: $E^{\mathsf{BJ}}/\mathbb{F}_s$ defined by equation $E^{\mathsf{BJ}} : 168700x^2 + y^2 = 1 + 168696x^2y^2$, with subgroup $\mathbb{G}^{\mathsf{BJ}}$ of prime order t.
- BW6-761 curve: $E^{\mathsf{BW}}/\mathbb{F}_u$ defined by equation $E^{\mathsf{BW}} : y^2 = x^3 - 1$, with subgroups $\mathbb{G}_1^{\mathsf{BW}}, \mathbb{G}_2^{\mathsf{BW}}$ of prime order v, and an efficiently computable pairing $e : \mathbb{G}_1^{\mathsf{BW}} \times \mathbb{G}_2^{\mathsf{BW}} \to \mathbb{G}_T^{\mathsf{BW}}$, where $\mathbb{G}_T^{\mathsf{BW}} \subset \mathbb{F}_{v^6}$ and has order v as well.
- BLS12-377 curve: $E^{\mathsf{BLS}}/\mathbb{F}_v$ defined by equation $E^{\mathsf{BLS}} : y^2 = x^3 + 1$, with subgroups $\mathbb{G}_1^{\mathsf{BLS}}, \mathbb{G}_2^{\mathsf{BLS}}$ of prime order w, and an efficiently computable pairing $e : \mathbb{G}_1^{\mathsf{BLS}} \times \mathbb{G}_2^{\mathsf{BLS}} \to \mathbb{G}_T^{\mathsf{BLS}}$, where $\mathbb{G}_T^{\mathsf{BLS}} \subset \mathbb{F}_{w^{12}}$ and has order w.

Finite fields.

- $\mathbb{F}_p$: base field of the SECP256K1 curve.
- $\mathbb{F}_q$: scalar field of $\mathbb{G}^{\mathsf{SEC}}$.
- $\mathbb{F}_r$: base field of the BN254 curve.
- $\mathbb{F}_s$: scalar field of $\mathbb{G}_1^{\mathsf{BN}}, \mathbb{G}_2^{\mathsf{BN}}, \mathbb{G}_T^{\mathsf{BN}}$ and base field of the BabyJubjub curve.
- $\mathbb{F}_t$: scalar field of $\mathbb{G}^{\mathsf{BJ}}$.
- $\mathbb{F}_u$: base field of the BW6-761 curve.
- $\mathbb{F}_v$: scalar field of $\mathbb{G}_1^{\mathsf{BW}}, \mathbb{G}_2^{\mathsf{BW}}, \mathbb{G}_T^{\mathsf{BW}}$ and base field of the BLS12-377 curve.
- $\mathbb{F}_w$: scalar field of $\mathbb{G}_1^{\mathsf{BLS}}, \mathbb{G}_2^{\mathsf{BLS}}, \mathbb{G}_T^{\mathsf{BLS}}$.

Generators.

We denote by G^{SEC} the generator of $\mathbb{G}^{\mathsf{SEC}}$ as defined in [7], G^{BN} the generator of $\mathbb{G}_1^{\mathsf{BN}}$ as defined in [17], G^{BJ} the generator of $\mathbb{G}^{\mathsf{BJ}}$ as defined in [16], G^{BW} the generator of $\mathbb{G}_1^{\mathsf{BW}}$ as defined in [11], and G^{BLS} the generator of $\mathbb{G}_1^{\mathsf{BLS}}$ as defined in [15].

- $G^{\mathsf{SEC}} =$ (0x79be667ef9dcbbac55a06295ce870b07029bfcdb2dce28d959f2815b16f81798, 0x483ada7726 a3c4655da4fbfc0e1108a8fd17b448a68554199c47d08ffb10d4b8).
- $G^{\mathsf{BN}} =$ (0x01, 0x02).
- $G^{\mathsf{BJ}} =$ (0x0b9feffffffffaaabffffffffffffffefffffffffffffffffffffffffffffff, 0x17fffffffff ffffffffffffffffffffffffffffffffffffffebaaedce6af48a03bbfd25e8cd0364141).
- $G^{\mathsf{BW}} =$ (0x1075b020ea190c8b277ce98a477beaee6a0cfb7551b27f0ee05c54b85f56fc779017ffac15520 ac11dbfcd294c2e746a17a54ce47729b905bd71fa0c9ea097103758f9a280ca27f6750dd0356133 e82055928aca6af603f4088f3af66e5b43d, 0x58b84e0a6fc574e6fd637b45cc2a420f952589884 c9ec61a7348d2a2e573a3265909f1af7e0dbac5b8fa1771b5b806cc685d31717a4c55be3fb90b6f c2cdd49f9df141b3053253b2b08119cad0fb93ad1cb2be0b20d2a1bafc8f2db4e95363).
- $G^{\mathsf{BLS}} =$ (0x008848defe740a67c8fc6225bf87ff5485951e2caa9d41bb188282c8bd37cb5cd5481512ffcd3 94eeab9b16eb21be9ef, 0x01914a69c5102eff1f674f5d30afeec4bd7fb348ca3e52d96d182ad44 fb82305c2fe3d3634a9591afd82de55559c8ea6).

4.2 Hash functions

DAVINCI uses different hash functions. On the one side, we use Keccak256 [4] for Ethereum address derivation, and Poseidon [9], MiMC and MiMC-7 [1] for arithmetic circuits within zero-knowledge proofs.

Keccak256. Keccak256 is the standard hash function used by Ethereum and is employed in DAVINCI to compress messages for signing and deriving Ethereum addresses from public keys over $\mathbb{G}^{\mathsf{SEC}}$. Keccak256 takes arbitrary-length bitstrings and outputs 256-bit digests: $\mathsf{Hash}_1 : \{0,1\}^* \to \{0,1\}^{256}$. This function is not SNARK-friendly and it is used exclusively in the authentication circuit, where the verification of Ethereum-compatible signatures requires reproducing the original message hash computed by Ethereum clients (see Section 5.4.2).

Poseidon. Poseidon is a SNARK-optimized hash function designed for efficient implementation inside arithmetic circuits. It is used in DAVINCI for computing commitments, nullifiers, and the state Merkle tree roots. Poseidon is used as $\mathsf{Hash}_2 : \mathfrak{F}_s \to \mathbb{F}_s$, where $\mathfrak{F}_s$ is the set of tuples of $\mathbb{F}_s$-elements of any length, and $\mathbb{F}_s$ is the field described in Section 4.1.

MiMC-7. Finally, we use functions from the MiMC family. On the one side, MiMC-7 hash function $\mathsf{Hash}_3 : \mathbb{F}_p \to \mathbb{F}_p$, where $\mathbb{F}_p$ is the set of tuples of $\mathbb{F}_p$-elements of any length, and $\mathbb{F}_p$ is the field described in Section 4.1. This hash is used to hash the public inputs of the different circuits and verify the associated proofs more efficiently (see Section 5.4 for further details). On the other side, MultiMiMC7. Different curves.

For the sake of clarity, in our notation we admit anything as an input of the above hash functions, although it might not be parsed as a sequence of field elements. Any bitstring can be converted to a tuple of field elements, so this does not change the actual working of the hash computation. In this document, it is implied that the proper conversion happens before feeding the input into the actual hash function.

4.3 Encryption schemes

To preserve ballot secrecy while enabling tallying, DAVINCI employs a threshold variant of the ElGamal cryptosystem instantiated over the BabyJubjub curve [14]. This scheme offers two properties crucial for the protocol: *additive homomorphism*, which allows the aggregation of encrypted votes without decryption, and *re-encryption*, which enables ciphertext randomization without altering the underlying plaintext. Together, these properties allow sequencers to tally votes while preventing voters from producing receipts that could be used for coercion or vote selling. The corresponding public key is generated collectively by the sequencers via the distributed key generation protocol described in Section 4.5, ensuring that no single entity can decrypt ballots unilaterally.

Encryption and decryption. Given a message m, the ElGamal encryption algorithm maps it into a group element M of $\mathbb{G}^{\mathsf{BJ}}$ and outputs a ciphertext as follows:

- $\mathsf{Enc}_{\mathsf{pk}}(m; k \in \mathbb{F}_t)$:
  1. Map m to $\mathbb{G}^{\mathsf{BJ}}$ via $M = m \cdot G^{\mathsf{BJ}}$.
  2. Compute $C_1 = k \cdot G^{\mathsf{BJ}} \in \mathbb{G}^{\mathsf{BJ}}$.
  3. Compute $C_2 = k \cdot \mathsf{pk} + M \in \mathbb{G}^{\mathsf{BJ}}$.
  4. Output $\mathsf{ciphertext} = (C_1, C_2) \in (\mathbb{G}^{\mathsf{BJ}})^2$.
- $\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ciphertext})$:
  1. Parse $\mathsf{ciphertext} = (C_1, C_2)$.
  2. Compute $K = \mathsf{sk} \cdot C_1 \in \mathbb{G}^{\mathsf{BJ}}$.
  3. Output $M = C_2 - K$.

Since the plaintext message space is typically small, recovering m from M is efficient: although the mapping $m \mapsto M$ is not generally invertible, it can be reversed by brute-force search or optimized techniques such as baby-step giant-step [5].

Homomorphic addition and reencryption. ElGamal encryption is additively homomorphic. Given two ciphertexts (C_1, C_2) and (C_1', C_2'), their component-wise addition yields another valid ciphertext $(C_1 + C_1',\ C_2 + C_2')$. The resulting ciphertext decrypts to the sum of the two underlying messages. This property allows sequencers to aggregate encrypted ballots directly. Moreover, to prevent linkability and ensure receipt-freeness, sequencers also re-randomize ciphertexts. Re-encryption exploits this property by adding to a ciphertext an encryption of zero. This operation yields a fresh ciphertext of the same message under new randomness, making it computationally infeasible to link the re-encrypted ballot with the original submission.

- $\mathsf{EncAdd}_{\mathsf{pk}}(\mathsf{ciphertext} \in (\mathbb{G}^{\mathsf{BJ}})^2, \mathsf{ciphertext}' \in (\mathbb{G}^{\mathsf{BJ}})^2)$:
  1. Parse $\mathsf{ciphertext} = (C_1, C_2) \in (\mathbb{G}^{\mathsf{BJ}})^2$.
  2. Parse $\mathsf{ciphertext}' = (C_1', C_2') \in (\mathbb{G}^{\mathsf{BJ}})^2$.
  3. Output $(C_1 + C_1', C_2 + C_2') \in (\mathbb{G}^{\mathsf{BJ}})^2$.
- $\mathsf{ReEnc}_{\mathsf{pk}}(\mathsf{ciphertext} \in (\mathbb{G}^{\mathsf{BJ}})^2; k \in \mathbb{F}_t)$:
  1. Parse $\mathsf{ciphertext} = (C_1, C_2) \in (\mathbb{G}^{\mathsf{BJ}})^2$.
  2. Compute $\mathsf{ciphertext}' = \mathsf{Enc}_{\mathsf{pk}}(0; k)$.
  3. Output $\mathsf{EncAdd}_{\mathsf{pk}}(\mathsf{ciphertext}, \mathsf{ciphertext}')$.

In the protocol, re-encryption is applied not only to newly submitted ballots but also to ballots already stored in the state tree. By refreshing the randomness of both new and existing ciphertexts, sequencers ensure that it is indistinguishable whether a ballot has been overwritten or merely re-randomized. This mechanism is essential for guaranteeing receipt-freeness: voters cannot produce a verifiable receipt of their choice, and adversaries cannot detect or prove whether a particular vote has been replaced (see Section 8).

4.4 Digital signature schemes

DAVINCI uses the elliptic curve digital signature algorithm (ECDSA) [13] over the SECP256K1 curve to ensure compatibility with standard Ethereum wallets. Verification of ECDSA signatures is performed inside zero-knowledge proofs using a specialized circuit that emulates SECP256K1 arithmetic. This approach is necessary because SECP256K1 is defined over a 256-bit prime field that differs from the native field used in the ZK-SNARK circuit (see Section5.4 for more details). Below, we describe the algorithms, which follow the standard ECDSA protocol. (The output of the hash does not match – check where variables live.)

- $\mathsf{S.Sign}_{\mathsf{sk}}(\mathsf{message} \in \{0,1\}^\star)$:
  1. Compute $h = \mathsf{Hash}_1(\mathsf{message}) \bmod q$.
  2. Select a random scalar $k \xleftarrow{\$} \mathbb{Z}_q^*$.
  3. Compute $R = k \cdot G^{\mathsf{SEC}} = (x_R, y_R) \in \mathbb{G}^{\mathsf{SEC}}$.
  4. Set $r = x_R \bmod q$.
     - If $r = 0$, go back to step 2.
     - Else, continue.
  5. Compute $s = k^{-1} \cdot (h + r \cdot \mathsf{sk}) \bmod q$.
     - If $s = 0$, go back to step 2.
     - Else, continue.
  6. Output $\sigma = (r, s)$.
- $\mathsf{S.Verify}_{\mathsf{pk}}(\mathsf{message} \in \{0,1\}^\star, \sigma = (r,s) \in (\mathbb{Z}_q^*)^2)$:
  1. Check that $\mathsf{pk} \in \mathbb{G}^{\mathsf{SEC}}$.
  2. Compute $h = \mathsf{Hash}_1(\mathsf{message}) \bmod q$.
  3. Compute $w = s^{-1} \bmod q$.
  4. Compute $u_1 = h \cdot w \bmod q$.
  5. Compute $u_2 = r \cdot w \bmod q$.
  6. Compute
  $$X = u_1 \cdot G^{\mathsf{SEC}} + u_2 \cdot \mathsf{pk} = (x_X, y_X) \in \mathbb{G}^{\mathsf{SEC}}.$$
  7. Accept if $r = x_X \bmod q$; otherwise, reject.

4.5 Key generation schemes

To ensure that no single entity controls the decryption key, DAVINCI employs a distributed key generation (DKG) protocol to jointly derive the ElGamal encryption key pair used for ballots encryption [?]. Sequencers who participate in the DKG ceremony are each identified by a unique index i. The outcome is a collective public key (`encryptionKey`) that is published on-chain and used by voters to encrypt their ballots, while the corresponding private key is secret-shared among the sequencers. Only a threshold number t out of n sequencers can later collaborate to decrypt the tally. Unlike classical DKG protocols where shares are exchanged in the clear, our construction uses encrypted shares and ZK-SNARK proofs of correctness. This allows the Ethereum smart contract to verify compliance without learning any of the underlying secrets, ensuring both security and verifiability in a fully decentralized setting.

Let t be the threshold parameter and n the number of sequencers, with $t \leq n$. Let G be a generator of the elliptic curve group of order q. Each participant P_i runs the following procedure.

- $\mathsf{GenerateEncryptedShares}(i)$:
  1. Select random scalars $a_{i,0}, \ldots, a_{i,t-1} \xleftarrow{\$} \mathbb{Z}_q$.
  2. Define the polynomial $f_i(x) = \sum_{j=0}^{t-1} a_{i,j} x^j$.
  3. For each $j \in \{0, \ldots, t-1\}$, compute public commitments $C_{i,j} = a_{i,j} \cdot G$.
  4. For each sequencer P_ℓ with index $\ell \in \{1, \ldots, n\}$, compute the secret share $s_{i,\ell} = f_i(\ell)$.

5. Encrypt each $s_{i,\ell}$ under P_ℓ's public key using a simplified version of the EC integrated encryption scheme (ECIES), obtaining $EncECIES(s_{i,\ell})$.
6. Output $\{C_{i,j}\}_{j=0}^{t-1}$ and $\{EncECIES(s_{i,\ell})\}_{\ell=1}^{n}$, together with a ZK-SNARK proof attesting that the encrypted values are consistent with the commitments.

The smart contract verifies the encrypted shares.

- VerifyEncryptedShare$(i, \ell, EncECIES(s_{i,\ell}), \{C_{i,j}\})$:
  1. Verify that the ZK-SNARK proof attached to $EncECIES(s_{i,\ell})$ is correct.
  2. If the proof is valid, the share is accepted; otherwise, the participant P_i may be slashed.

After collecting valid encrypted shares, each sequencer P_j can recover their secret share s_j by decrypting all contributions addressed to them:

- DeriveSecretShare(j):
  1. Decrypt all $EncECIES(s_{i,j})$ values received from other participants.
  2. Compute $s_j = \sum_{i=1}^{n} s_{i,j} \mod q$.
  3. Output s_j.

- DerivePublicKey$(\{C_{i,0}\}_{i=1}^{n})$:
  1. Compute $pk = \sum_{i=1}^{n} C_{i,0}$.
  2. Output pk.

  Note that $C_{i,0} = a_{i,0} \cdot G$, so the collective public key corresponds to $pk = s \cdot G$, where $s = \sum_{i=1}^{n} a_{i,0} \mod q$ is the collective private key, unknown to any single participant.

This approach ensures that the encryption public key is securely generated in a decentralized way, that each sequencer learns only their own secret share, and that the correctness of the entire DKG procedure is verifiable on-chain. Misbehavior, such as submitting invalid shares, can be detected and penalized through the slashing mechanism enforced by the smart contract (see Section 7).

4.6 Merkle trees

The DAVINCI protocol uses sparse Merkle Trees (SMTs) as its primary data structure for maintaining off-chain state. Following the construction in [2], these are full binary trees of fixed depth in which each leaf encodes a key-value pair and empty leaves are explicitly represented, allowing for both inclusion and non-inclusion proofs. A key property of SMTs is that the hash of each key determines a unique traversal path from the root to the leaf, regardless of insertion order. When inserting a new key, the tree descends until it encounters either an empty node or a leaf. In case of a path collision, the algorithm splits at the first differing bit, creating intermediate nodes accordingly.

In DAVINCI, SMTs are used to represent two core data structures: the census tree, which encodes the list of eligible voters along with their respective voting weights, and the state tree, which encodes the current status of submitted votes. Both trees have fixed depth 64. The census tree is instantiated with the MiMC hash function over the BLS12-377 scalar field, while the state tree uses Poseidon over the BN254 scalar field. Only the root hash of each tree is published on-chain, ensuring verifiability with minimal on-chain storage. To support state transitions and circuit verification, each tree MT has the following functions associated:

- MT.Root(): it returns the current Merkle root of the tree.
- MT.Insert(path, leaf): it inserts a new leaf leaf at the position defined by the given path.
- MT.Update(leaf_{old}, leaf): it updates an existing leaf in the tree.
- MT.MembershipProof(leaf): it returns a Merkle proof of inclusion for the given leaf.
- MT.NonMembershipProof(leaf): it returns a proof that the given leaf is not included in the tree.
- MT.Verify(leaf, proof, root): it verifies that the given proof corresponds to the claimed root and leaf.

These functions are used by sequencers to construct proofs that certify the validity of state transitions. In particular, they allow for off-chain state evolution that is independently verifiable on-chain and within ZK circuits. Additional details on the structure of the census and state trees, as well as the encoding of their leaves, are provided in Section 5.2.

4.7 Zero-knowledge proof systems

Zero-knowledge succinct non-interactive arguments of knowledge (ZK-SNARKs) are a crucial component in ensuring the integrity of the voting process. Voters generate ZK-SNARK proofs to demonstrate that their encrypted ballots comply with the rules and constraints defined by the election parameters, without revealing any information about their choices. Similarly, sequencers produce proofs to certify the correctness of vote aggregation and state transitions throughout the protocol. P.Prove (), P.Verify (proof, PI) – pkey, vkey.

All ZK circuits in DAVINCI are compiled using the Groth16 proof system [10], a widely adopted ZK-SNARK construction known for its succinctness, efficient verification, and minimal proof size. Although all circuits rely on the same proving system, they are instantiated with different elliptic curves depending on the cryptographic requirements of each phase. Specifically, the vote validity circuit is compiled over the BN254 curve, the census-related circuit uses BLS12-377, and the aggregation circuit is instantiated over BW6-761. Finally, the last circuit used by the sequencer—responsible for generating the final proof of correct tallying and result—is compiled over BN254 as well, since this is the proof that is verified on-chain by the smart contract using Ethereum's native precompiles. Further details on the circuits are provided in Section 5.4 and a summary of the instantiations can be found in Figure 3.

5 Voting protocol

This section outlines the DAVINCI voting protocol. We introduce the parties involved in the process in Section 5.1. In Section 5.2, we describe the census and state Merkle trees, and in Section 5.3 the Ethereum smart contracts. In Section 5.4, we present the ZK circuits used to enforce vote validity, authentication, aggregation, and state transitions. Finally, in Section 5.5, we detail the full protocol flow step by step, from the voting setup to the results validation and process finalization.

5.1 Parties involved

A voting process involves three types of participants: the organizer, the sequencer, and the voters.

Organizer. The organizer is the entity responsible for defining and setting up the voting process. Its key responsibilities include defining the voting parameters and gathering the census data. The organizer ensures that the election is structured correctly but does not participate in vote collection or tallying.

Sequencer. Sequencers are a set of decentralized parties that facilitate the voting process. On the one hand, they collaboratively generate a public encryption key that voters use to encrypt their votes. Then, during the voting phase, they receive and verify encrypted votes from voters and reencrypt votes to prevent coercion.

Voter. Voters are the participants that belong the census that cast their votes in the election.

5.2 Merkle trees

Merkle trees are used to create a cryptographic representation of both the voter census registry and the voting process state.

5.2.1 Census tree

The census Merkle tree, denoted by $\mathsf{MT}^{\mathtt{census}}$ encodes the list of eligible voters and their associated voting weights. Formally, it is implemented as a sparse Merkle tree (SMT) of fixed depth[4], as described in Section 4.6.

[4]In the present work, all circuits are instantiated using sparse Merkle trees as described above. However, the protocol does not strictly depend on this structure: any authenticated data structure that supports efficient commitments and membership proofs verifiable inside zero-knowledge circuits could be employed. Adapting the protocol to such alternatives would only require modifying the corresponding circuits, while the overall flow of the voting process would remain unchanged.

Each voter is mapped to a unique path in the tree, typically derived from their Ethereum address or another unique identifier, and the corresponding leaf stores the voting weight of that address. While the default weight is usually one, the system also supports weighted voting, where different participants may have different levels of influence.

To construct the census, the organizer collects the dataset of eligible voters and their weights, computes the corresponding tree, and publishes its root (`censusRoot`) on-chain. The full dataset does not need to be stored on-chain; instead, it is distributed off-chain (e.g., via IPFS or another content-addressed storage system), enabling verifiers and sequencers to independently reconstruct the tree and produce membership proofs when needed. This design ensures transparency and verifiability while minimizing on-chain storage requirements.

The census tree plays a central role during the voting phase. Voters must provide a Merkle proof showing that their address and weight are included in the tree committed by the organizer. These proofs are then verified within ZK circuits, which guarantees that only eligible participants can cast ballots without requiring the disclosure of the entire census. In this way, the census Merkle tree provides a compact, cryptographically verifiable commitment to voter eligibility that supports both privacy and scalability.

Although the protocol does not mandate a specific method for constructing the census tree, different instantiations are possible depending on the application context:

- *Private datasets.* Organizations maintaining internal voter registries (e.g., membership databases or CSV files) can directly convert them into the Merkle format. This approach keeps the registry confidential, with only the root hash made public.
- *Self-sovereign identity (SSI).* If voters manage identities through SSI systems, the organizer can aggregate these identities and build the tree accordingly. This enables participation without requiring centralized identity management.
- *Ethereum-based tokens.* Token balances (e.g., ERC-20 or NFTs) may also define eligibility. In this setting, the census can either be created via the optimistic approach, taking a snapshot of token balances and allowing a dispute period for fraud proofs, or through a ZK-SNARK that proves correctness directly from Ethereum storage proofs and ensures that both individual balances and the token's total supply are consistent.

This flexibility allows the protocol to accommodate a wide range of use cases, from private associations to decentralized communities, while maintaining the same security and verifiability guarantees.

5.2.2 State tree

The state Merkle tree, denoted by $\mathsf{MT}^{\mathtt{state}}$, represents the evolving state of the voting process. Like the census tree, it is implemented as a SMT of fixed depth, as described in Section 4.6. In contrast to the census tree, which remains static once the election begins, the state tree is dynamic: it is updated throughout the process as new votes are cast, validated, and incorporated by the sequencers.

The leaves of the state tree store the global process parameters, the encrypted ballots, and the set of vote identifiers, as illustrated in Figure 2. More specifically:

Reserved addresses for metadata. The first six addresses are reserved for the process parameters as follows:

- `0x0`: it stores a unique identifier for the voting process (`processID`).
- `0x1`: it stores the committed census root (`censusRoot`).
- `0x2`: it stores the ballot mode configuration, which is the set of rules for validating votes (`ballotMode`).
- `0x3`: it stores the encryption public key used to encrypt the votes (`encryptionKey`).
- `0x4`: it stores an accumulator of the encrypted votes that need to be added to the tally (`resultsAdd`).
- `0x5`: it stores an accumulator of the encrypted votes that need to be substracted from the tally because they have been overwritten (`resultsSub`).

Encrypted ballots. Each voter's encrypted ballot is stored in the state tree at a leaf whose path corresponds to their Ethereum address (or another unique identifier). The leaf value is the ciphertext corresponding to the ballot generated under the election's public encryption key. If the voter later overwrites their ballot, the same path is updated with the new ciphertext.

Vote identifiers. In addition to storing ballots, the state tree also tracks the set of vote identifiers. Each vote identifier `voteID`, which is derived from the `processID`, the voter's address, and fresh randomness, determines the path to a dedicated leaf. Initially empty, these leaves are filled when the corresponding `voteID` is used, thereby recording that it has been consumed. This enables efficient non-membership proofs: before accepting a new ballot, the sequencer checks that the corresponding `voteID` path is still empty, and upon inclusion, the leaf is marked as used to prevent reuse.

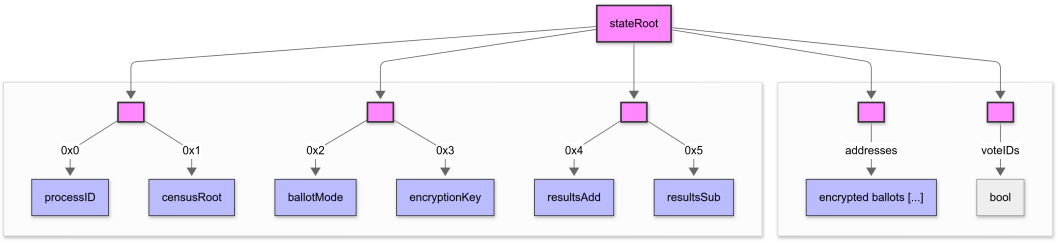

Fig. 2. Structure of the state Merkle tree. The first leaves are reserved for global process parameters while subsequent leaves store encrypted ballots indexed by voter addresses and vote identifiers.

At the beginning of the election, the state tree is initialized with the process parameters and empty accumulators, and its root (`stateRoot`) is published on-chain. As the voting progresses, sequencers update the tree whenever new votes are received, producing a new root for each transition. To guarantee correctness, each update is accompanied by a ZK-SNARK proof attesting that the transition from the previous root to the new root respects all protocol constraints: ballots are well-formed and comply with the voting rules, voters are eligible according to the census, the `voteID` non-membership condition prevents double voting (or overwrites are handled correctly), and encrypted results are aggregated consistently. In addition, sequencers reencrypt stored ciphertexts, including both newly submitted and already included ballots. This mechanism refreshes the randomness of the encrypted votes without altering their underlying content, ensuring receipt-freeness and concealing whether a particular ballot has been overwritten or simply re-randomized.

In this way, the state tree provides a compact and tamper-proof commitment to the current status of the election. At the end of the process, the final state root together with the on-chain verification of all transitions serves as a complete cryptographic record of the election, encapsulating both the tally and the integrity of the entire voting procedure.

5.3 Smart contracts

An Ethereum compatible network is used as the source of truth for the voting system. By leveraging an Ethereum virutal machine (EVM) blockchain, the voting process management ensures that all transitions are immutable and verifiable by all participants. To this end, we implement the following three smart contracts.

Process management smart contract. It is responsible for the life-cycle management of voting processes. It includes the initiation, monitoring, execution, and closure of voting events.

Sequencer registry smart contract. It keeps track of the existing available sequencers, stores the collateral of the sequencers to ensure (that incentivizes) good behaviour, and is used to coordinate the distributed key generation when a new voting process is created.

Results verification smart contract. For each voting process, this smart contract maintains the integrity of the cast votes and process life cycle. It verifies that each state transition committed by a sequencer adheres to the predefined rules.

5.4 Circuits

At the core of DAVINCI lie a set of arithmetic circuits that enforce the correctness of every step of the voting process. Each circuit corresponds to a distinct task and, taken together, they guarantee that all protocol rules are satisfied without revealing any private information. Specifically, the *ballot circuit* (Section 5.4.1) ensures that encrypted votes are valid and well-formed, the *verifier circuit* (Section 5.4.2) verifies voter eligibility and census membership; the *aggregation circuit* (Section 5.4.3) combines multiple authenticated votes into a single proof; and the *state transition circuit* (Section 5.4.4) updates the global state root and produces the ZK-SNARK proof that is verified on-chain. The flow of these circuits and their interactions is shown in Figure 3.

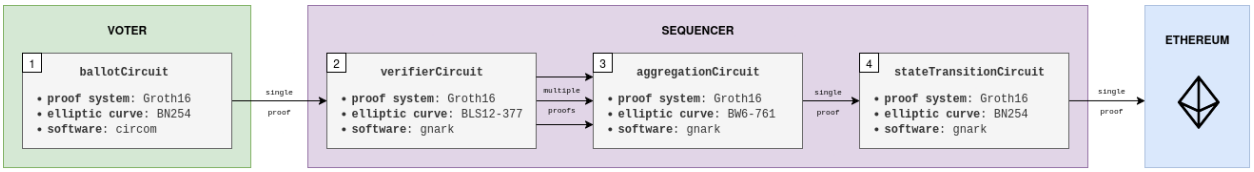

Fig. 3. Overview of the circuit flow in DAVINCI.

For clarity, the circuits are described in this section with their full set of public inputs explicitly listed. In the actual implementation, however, we apply an optimization: rather than exposing all public inputs individually, we compute a single commitment $h = Hash(PI)$ and use h as the only public input. The proof verifier then checks that $Hash(PI) = h$, ensuring that the original public inputs are consistent while reducing both proof size and verification cost. In DAVINCI, this commitment is computed using the MiMC-7 hash function (use hlget to point "Hash" to MiMC-7). Note that this optimization does not alter the semantics of the circuits but significantly reduces verifier complexity and on-chain verification costs. Other implementation aspects such as the exact number of constraints and the proving frameworks used are deferred to Section 8.2.

5.4.1 Ballot circuit

The ballot circuit, illustrated in Figure 4, is generated locally by the voter at the time of casting a ballot. Its purpose is twofold: first, to prove that the encrypted ballot is valid and complies with the protocol rules defined by the organizer; and second, to prove that the vote identifier (`voteID`) has been correctly derived. The correctness of this circuit is attested by a ZK proof generated using the ZK-SNARK protocol described in Section 4.7, instantiated over the BN254 curve. We call the resulting proof *ballot proof* and denote it as `ballotProof`. We detail below the inputs and constraints of the ballot circuit.

Public inputs

- public `processID`: voting process identifier.
- public `ballotMode`: ballot protocol configuration.S
- public `encryptionKey` (EPK): encryption public key.
- public `address`: voter's address.
- public `weight`: voter's weight.
- public (C_1, C_2): encrypted ballot.
- public `voteID`: vote's unique identifier.

Private inputs

- private `ballot`: plaintext voter's voting choice.
- private k: random scalar.

Constraints

- *Vote encryption.* Correct encryption of the ballot: $(C_1, C_2) = \mathsf{Enc}_{\texttt{encryptionKey}}(\texttt{ballot}; k)$.
- *Protocol rules compliance.* The ballot meets the protocol rules (number of valid choices, etc.): ...+`weight`.
- *Vote identifier.* The vote's identifier is correctly computed: $\texttt{voteID} = \text{Hash}(\texttt{processID} \,||\, \texttt{address} \,||\, k)$.

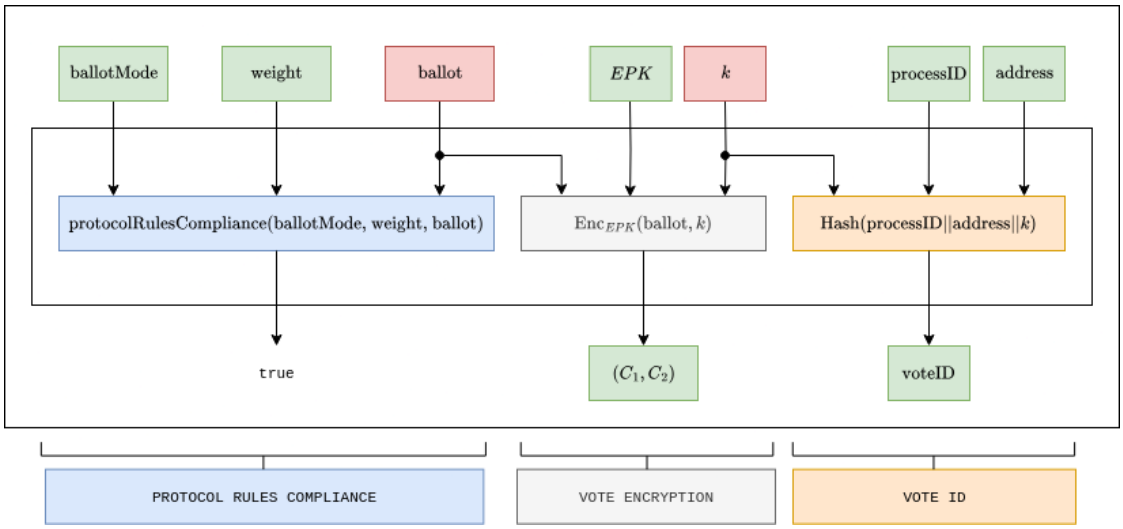

Fig. 4. Ballot circuit. All public values are framed in green.

5.4.2 Verifier circuit

The verifier circuit, illustrated in Figure 5, is generated by the sequencer when processing a vote. This circuit verifies the ballot's proof generated by the voter and it also validates the voter's eligibility in the census and the correctness of their digital signature. The correctness of this circuit is attested by a ZK proof generated using the ZK-SNARK protocol described in Section 4.7, instantiated over the BLS12-377 curve. We call the resulting proof the *authentication proof* and denote it as `authenticationProof`. We detail below the inputs and constraints of this circuit. – Authentication of verification proof?

Public inputs

- public `processID`: process identifier.
- public `ballotMode`: ballot protocol configuration.
- public `encryptionKey` (EPK): encryption public key.
- public (C_1, C_2): encrypted ballot.
- public `voteID`: voter's identifier.
- public `censusRoot`: census Merkle tree root.
- public `pk`: voter's public key.

Private inputs

- private `ballotProof`: voter's ballot proof.
- private `weight`: voter's weight.
- private `censusProof`: census membership proof.
- private `signature`: voter's signature.

Constraints

- *Voter's proof verification.* The vote zkProof is valid for the inputs provided:

$$\mathsf{P.Verify}\left(\texttt{ballotProof}, PI = (\texttt{processID}, \texttt{ballotMode}, \texttt{encryptionKey}, (C_1, C_2), \texttt{voteID}, \texttt{weight})\right).$$

- *Census membership.* The address derived from the user's public key is part of the census, and verifies the census proof with the user's weight provided:

$$\mathsf{MT}^{\texttt{census}}.\mathsf{MembershipProof}(\texttt{weight}, \texttt{address} = \mathsf{Hash}_1(\mathsf{pk}), \texttt{censusProof}, \texttt{censusRoot}).$$

- *Authentication + non-malleability.* The signature of the vote's identifier is valid for the given public key:

$$\mathsf{S.Verify}_{\mathsf{pk}}(\texttt{voteID}, \texttt{signature}).$$

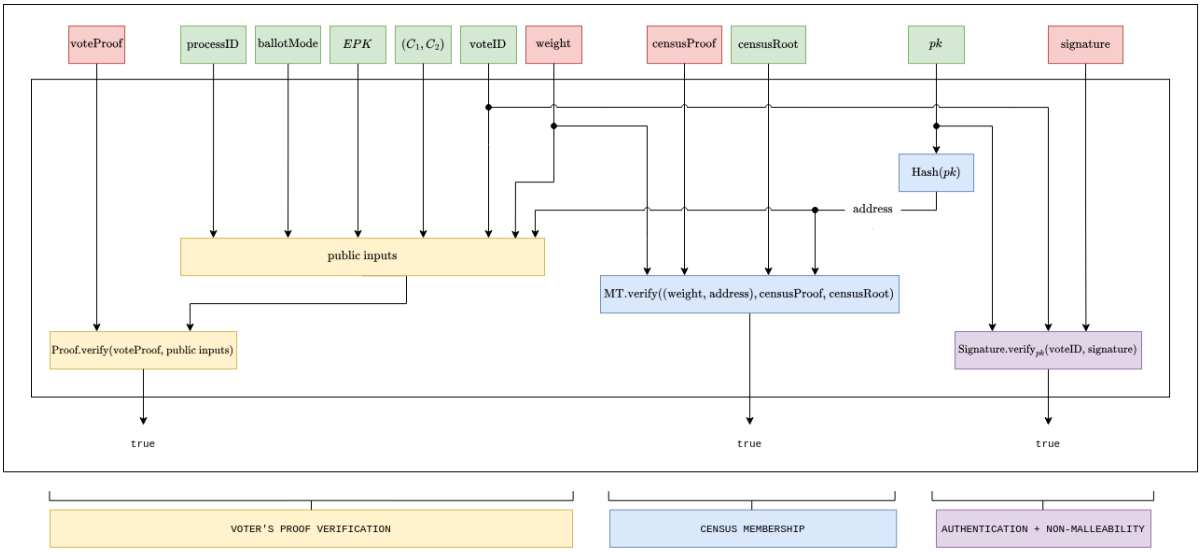

Fig. 5. Authentication circuit. All public values are framed in green.

5.4.3 Aggregation circuit

The aggregation circuit, illustrated in Figure 6, is generated by the sequencer to combine multiple authenticated votes into a single proof. Its purpose is to reduce verification overhead by recursively aggregating individual authentication proofs, while ensuring that all aggregated votes belong to the same voting process. The batch size, i.e., the maximum number of proofs aggregated in a single execution, is a fixed parameter (currently set to 60). If fewer proofs are available, the sequencer pads the batch with dummy proofs so that the circuit always operates on a fixed-size input. The correctness of this circuit is attested by a ZK proof generated using the ZK-SNARK protocol described in Section 4.7, instantiated over the BW6-761 curve. We call the resulting proof the *aggregation proof* and denote it as `aggregationProof`. We detail below the inputs and constraints of the aggregation circuit.

Public inputs

- public $(C_1, C_2)_i$: list of voters' encrypted ballots.
- public $\texttt{voteID}_i$: set of vote's identifiers.

- public pk_i: list of voters' public keys.
- public `processID`: process identifier.
- public `ballotMode`: ballot protocol configuration.
- public EPK: encryption public key.
- public `censusRoot`: census Merkle tree root.

Private inputs

- private $\texttt{authenticationProof}_i$: set of authentication proofs.

Constraints

- *Votes aggregation.* The accumulated authentication proofs are valid (for the given inputs). For every i:

  $$\mathsf{P.Verify}(\texttt{authenticationProof}, PI = ((C_1, C_2)_i, \texttt{voteID}_i, \mathsf{pk}_i, \texttt{processID}, \texttt{ballotMode}, EPK, \texttt{censusRoot})).$$

- *Shared public inputs.* The public inputs `processID`, `ballotMode`, `encryptionKey`, and `censusRoot` are the same for all proofs.

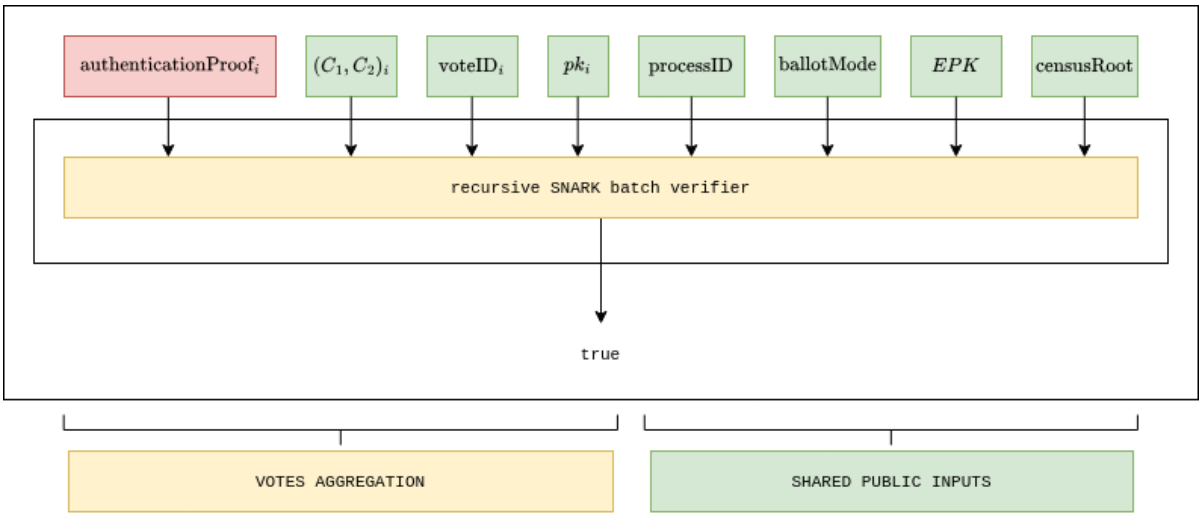

Fig. 6. Aggregation circuit. All public values are framed in green.

5.4.4 State transition circuit

The state transition circuit, illustrated in Figure 7, is generated by the sequencer to update the global state of the voting process. Its purpose is to verify that a batch of aggregated votes has been correctly incorporated into the state Merkle tree, that overwrites and vote identifiers are handled consistently, and that the accumulators of encrypted results are updated accordingly. The correctness of this circuit is attested by a ZK proof generated using the ZK-SNARK protocol described in Section 4.7, instantiated over the BN254 curve, which is supported by Ethereum's native precompiles for proof verification. We call the resulting proof the *state proof* and denote it as `stateProof`. This proof is submitted on-chain and verified by the process management smart contract, ensuring that every accepted state transition complies with the protocol rules.

To formalize the setting, let the sequencer collect a batch of N votes ($N < 60$), of which n are new submissions and $m = N - n$ are overwrites of previously cast ballots. In the following, we describe the inputs and constraints of the state transition circuit in detail.

Public inputs

- public $\texttt{stateRoot}^{old}$: current state tree root.
- public $\texttt{stateRoot}^{new}$: new tree root after all state transitions are applied.
- public $\texttt{numOverwrites}^{old}$: current number of overwrites.
- public $\texttt{numOverwrites}^{new}$: number of overwrites after all state transitions are applied.
- public $\texttt{numVotes}^{old}$: current number of votes.
- public $\texttt{numVotes}^{new}$: total number of votes after all state transitions are applied.
- public $\texttt{blobComm}$: commitment to the data blob.

Private inputs

- private $\{\texttt{aggregationProof}_i\}_{i=1}^{N}$: set of aggregation proofs.
- private $\texttt{censusRoot}$: census Merkle tree root.
- private $\texttt{ballotMode}$: ballot protocol configuration.
- private EPK: encryption public key.
- private $\texttt{processID}$: process identifier.
- private $\{\texttt{voteID}_i\}_{i=1}^{N}$: set of vote's identifiers.
- private $\{(C_1, C_2)_i\}_{i=1}^{N}$: set of voters' encrypted ballots.
- private $\{\mathsf{pk}_i\}_{i=1}^{N}$: list of voters' public keys.
- private $\{\texttt{merkleProofs}_i^{encBallots}\}_{i=1}^{N}$: Merkle tree proofs of encrypted ballots. If it is a new vote, the proof will be a non-inclusion proof, if it is an overwrite, an inclusion proof.
- private $\{\texttt{merkleProofs}_i^{\texttt{voteID}}\}_{i=1}^{N}$: non-membership proofs of vote identifiers.
- private $\texttt{resultsAdd}^{old}$: current accumulator of votes (before the transitions).
- private $\texttt{resultsSub}^{old}$: current accumulator of votes that have been overwritten (before the transition).
- private k: random scalar.
- private Census Merkle tree (old): all leaves of the old tree (except the first six leaves).

Constraints

- *Aggregation verification.* The accumulated authentication proofs are valid. For every $i \in \{1, \dots, N\}$:

  $$\mathsf{P.Verify}(\texttt{authenticationProof}, PI = ((C_1, C_2)_i, \texttt{voteID}_i, \mathsf{pk}_i, \texttt{processID}, \texttt{ballotMode}, EPK, \texttt{censusRoot})).$$

- *Transition verification.* Verify initial state and state transitions are applied correctly.
  - Verify that the inputs $\texttt{processID}$, $\texttt{censusRoot}$, $\texttt{ballotMode}$, $\texttt{encryptionKey}$, $\texttt{resultsAdd}$, and $\texttt{resultsSub}$ correspond to the content of the census Merkle tree leaves $\texttt{0x0}$, $\texttt{0x1}$, $\texttt{0x2}$, $\texttt{0x3}$, $\texttt{0x4}$, $\texttt{0x5}$, respectively. The root of the tree, with all data and those six leaves, results in $\texttt{stateRoot}$.

    $$\mathsf{MT}^{\texttt{state}}(\text{with these six leaves and the rest}).\mathsf{Root}() = \texttt{stateRoot}.$$

  - Verify that none of the $\texttt{voteID}_i$ for all $i \in \{1, \dots, N\}$ are initially part of the tree:

    $$\mathsf{MT}^{\texttt{state}}.\mathsf{NonInclusionVerify}(\texttt{voteID}_i, \texttt{merkleProofs}_i^{\texttt{voteID}}, \texttt{stateRoot}).$$

  - Ensure that new votes are not part of the tree yet. For $i = 1, \dots, n$:

    $$\mathsf{MT}^{\texttt{state}} - \text{the leaf following the } \texttt{address}_i = \mathsf{Hash}_1(\mathsf{pk}_i) \text{ path is empty.}$$

- Ensure that overwrites do correspond to votes that were already on the tree. For $i = n+1, \dots, N$:

$$\mathsf{MT}^{\mathtt{state}}\text{– the leaf following the } \mathtt{address}_i = \mathsf{Hash}_1(\mathsf{pk}_i) \text{ path stores an encrypted ballot.}$$

- State transitions.
  - For new votes (keep iterating for all of them):
    1. Add encrypted ballot to the tree: $\mathsf{MT}^{\mathtt{state}}.\mathsf{Insert}(\mathtt{address} = \mathsf{Hash}_1(\mathsf{pk}), (\mathsf{C}_1, \mathsf{C}_2))$.
    2. Add vote identifier: $\mathsf{MT}^{\mathtt{state}}.\mathsf{Insert}(\mathtt{voteID})$.
    3. Increase votes counter: $\mathtt{numVotes} = \mathtt{numVotes} + 1$.
    4. Aggregate the vote to the results accumulator: $\mathtt{resultsAdd} = \mathtt{resultsAdd} + (C_1, C_2)$.
  - For overwritten votes (keep iterating for all of them):
    1. Overwrite the encrypted ballot: $\mathsf{MT}^{\mathtt{state}}.\mathsf{Update}(\mathtt{address} = \mathsf{Hash}_1(\mathsf{pk}), (\mathsf{C}_1, \mathsf{C}_2))$.
    2. Add vote identifier: $\mathsf{MT}^{\mathtt{state}}.\mathsf{Insert}(\mathtt{voteID})$.
    3. Increase votes counter: $\mathtt{numVotes} = \mathtt{numVotes} + 1$.
    4. Increase overwrites counter: $\mathtt{numOverwrites} = \mathtt{numOverwrites} + 1$.
    5. Aggregate the vote to the results accumulator: $\mathtt{resultsAdd} = \mathtt{resultsAdd} + (C_1, C_2)$.
    6. Aggregate the old vote to the accumulator: $\mathtt{resultsSub} = \mathtt{resultsSub} + (C_1, C_2)^{old}$.

    The old ciphertexts $(C_1, C_2)^{old}$ should also be inputs or retrieved from the MT inclusion proofs.
- After all previous state transitions are done, recompute new tree root:

$$\mathtt{stateRoot}^{new} = \mathsf{MT}^{\mathtt{state}}.\mathsf{Root}().$$

- Reencrypt all leafs of the tree (both old and new ones). New are ok, but for those that are old, also verify MTProofs of membership?.

$$newLeafContent = \mathsf{ReEnc}_{\mathtt{encryptionKey}}((C_1, C_2)_i, k), \quad k = Hash(k).$$

- Data blob commitment. Calculation of the commitment to all leafs that changed in the tree after all transition states were applied.

$$\mathtt{blobComm} = Commitment(\text{all leaves that changed}).$$

5.4.5 Results circuit

It needs to be done.

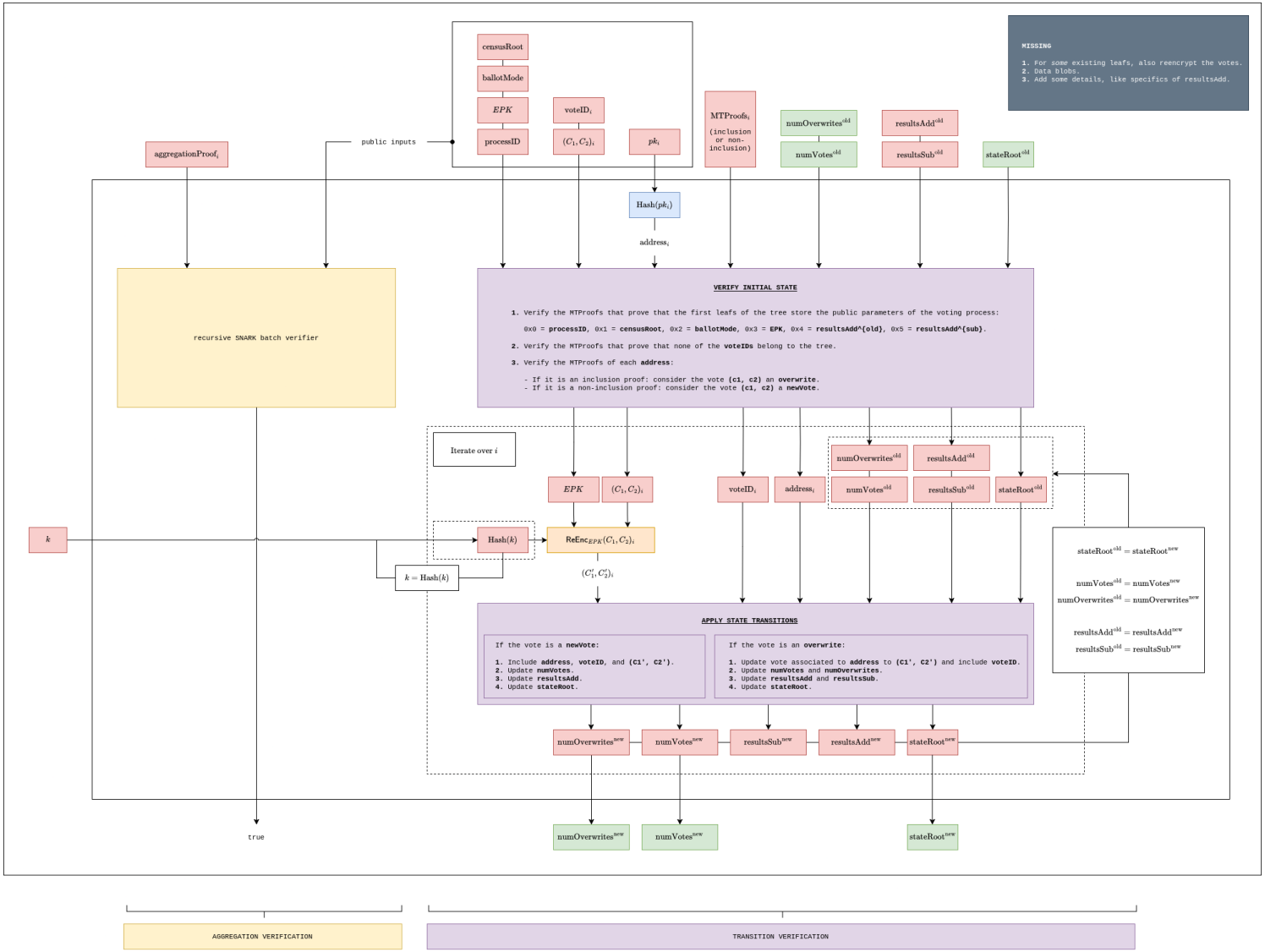

Fig. 7. State transition circuit. All public values are framed in green.

5.5 Protocol flow

In this section we describe the overall flow of the DAVINCI voting protocol. The process is structured into four main phases: voting setup, encryption key generation, votes collection, and results validation. Each phase specifies the interactions between the organizer, voters, sequencers, and the smart contracts that coordinate the voting process. An overview of the complete process is shown in Figure 8.

5.5.1 Voting setup

In this phase, sequencers register and the organizer sets the election up.

sequencers Sequencers must first register in the sequencer registry smart contract, a process that remains open continuously. Registration involves publishing their identity, network endpoint (e.g., URI), and a collateral deposit to ensure honest participation. The collateral can later be slashed in case of misbehavior.

organizer For each new election, the organizer performs the following steps:

1. Prepare the census data `censusData` and generate the census Merkle tree $\mathsf{MT}^{\mathtt{census}}$ with that data.
2. Compute the census commitment $\mathtt{censusRoot} = \mathsf{MT}^{\mathtt{census}}.\mathsf{Root}()$.
3. Define the election details, including duration, options, and the ballot mode `ballotMode`.
4. Compute a unique process identifier `processID`, which is a 32-byte number derived from the organizer's Ethereum address, nonce, and the chain ID.
5. Set security parameters for the DKG ceremony (e.g., timeout, minimum number of sequencers).
6. Submit all this information in a transaction to the process management smart contract (pay tx fees).

5.5.2 Encryption key generation

In this phase, sequencers collectively generate the public encryption key.

sequencers Sequencers fetch the process parameters (`processID`, `ballotMode`, `censusRoot`) from the process management smart contract and participate in a DKG protocol before the timeout set by the organizer expires. If the minimum number of sequencer contributions is reached, the public encryption key `encryptionKey` is derived and made available on-chain. This ensures that no single party controls the secret key, and that decryption later requires threshold participation (give more details).

Alternatively, generate a SNARK proving the EPK has been generated correctly and the contract verifies it.

5.5.3 Votes collection

In this phase, voters cast their ballots and send them to the sequencers, who collect them.

voter To cast a ballot, a voter does the following:

1. Select a sequencer from the sequencer registry smart contract.
2. Retrieve their census membership proof `censusProof` to prove eligibility.
3. Fetch parameters `encryptionKey`, `ballotMode`, and `processID` from the process management contract.
4. Select their ballot choice `ballot` according to the rules of `ballotMode`.
5. Sample fresh randomness $k \in \mathbb{F}$ and encrypt the ballot using `encryptionKey`:

$$(C_1, C_2) = \mathsf{Enc}_{\mathtt{encryptionKey}}(\mathtt{ballot}; k). \tag{1}$$

6. Use their Ethereum address `address` and the previous k to compute a unique vote identifier:

$$\mathtt{voteID} = \mathsf{MultiMiMC7}(\mathtt{processID} \,||\, \mathtt{address} \,||\, k). \tag{2}$$

7. Use the ballot circuit (Section 5.4.1) to generate a ZK-SNARK proof to prove that Eqs. (1) and (2) are correctly computed and that `ballot` satisfies ballot protocol rules according to `ballotMode`:

$$\texttt{voteProof} = \text{Prove}(\text{Circuit, PI} = ()).$$

8. Sign the vote identifier with their Ethereum secret key to authenticate the vote and ensure that was cast by a legitimate voter:

$$\texttt{signature} = \mathsf{S.Sign}_{\mathsf{sk}}(\texttt{voteID}).$$

9. Submits the package

$$\texttt{vote} = [\texttt{processID}, \texttt{voteID}, \texttt{encryptedBallot}, \texttt{censusProof}, \texttt{voteProof}, \texttt{signature}]$$

to the chosen sequencer.

`sequencers` Upon receiving votes, a sequencer does the following:

1. Retrieve the current state root `stateRoot` and census root `censusRoot` from the process management smart contract and the associated data from the Ethereum data blobs.
2. Verify each submission by generating a state transition proof (see Section 5.4.4), ensuring:
   - the validity of the voter's zkSNARK proof,
   - correct accumulation of encrypted votes via ElGamal's homomorphic properties,
   - eligibility of voters (via census Merkle proofs),
   - absence of double voting (or correct handling of overwrites via nullifiers),
   - consistency between data blobs and the on-chain state.
3. Submit the updated state root to the smart contract, while the full data are stored in Ethereum blobs.

Sequencers repeat this process until the voting deadline set by the organizer.

5.5.4 Results validation

After the voting period expires, t out of n sequencers publish their decryption shares of the election private key. Once the threshold is reached, the election secret key can be reconstructed (on-chain or off-chain), enabling the decryption of the aggregated tally.

Rewards and penalties for sequencers are managed according to their correct participation: sequencers are rewarded proportionally to the number of votes sequenced, and may be slashed for failing to provide a valid decryption share.

At this point:

- Anyone can verify the correctness of the final results by checking the zkSNARK state proofs and on-chain commitments.
- The combination of the final state root, the decryption of the accumulators, and the publicly verifiable zkSNARK proofs guarantees the integrity of the entire voting process.

The process is considered finalized once the decrypted tally is available and all state transition proofs have been verified on-chain. At this stage, both the election results and the complete integrity of the process are permanently recorded and publicly auditable.

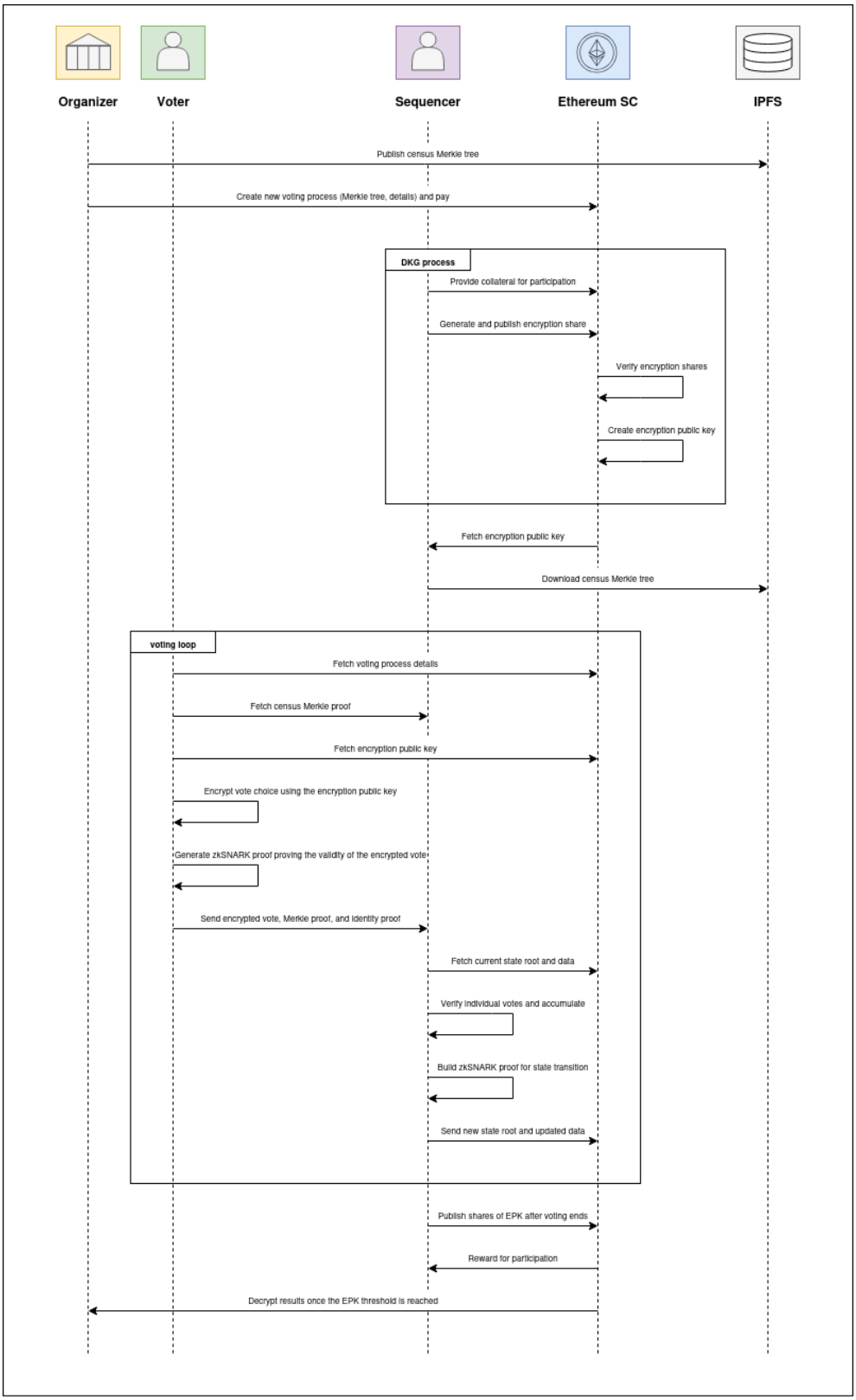

Fig. 8. Vocdoni voting process overview.

6 Ballot protocol

The ballot protocol provides a unified and parametric way to represent a wide range of voting systems within DAVINCI. Instead of designing a separate circuit for each voting rule, the protocol defines ballots as fixed-length arrays of integers, subject to a small set of configurable parameters. These parameters constrain the values that can appear in each field of the ballot and the aggregate properties of the ballot as a whole. By adjusting these parameters, the same circuit can implement approval voting, ranking, quadratic voting, multiple choice, and many other schemes. In the protocol we refer to the different configurations as the *ballot mode* (`ballotMode`). This abstraction has two key advantages. First, it allows a broad variety of voting systems to be supported with a minimal circuit design, avoiding conditional logic that would otherwise increase constraint counts. Second, it provides a common interface for tallying, since all ballots are aggregated into a single results array regardless of the voting mode.

Definition of parameters. Each ballot is defined as an array of integer values subject to a set of configurable parameters. These parameters are defined as follows:

- `numFields`: the maximum number of fields (i.e., options) in the ballot.
- `minValue`: the minimum value that each field in the ballot can take.
- `maxValue`: the maximum value that each field in the ballot can take.
- `uniqueValues`: a Boolean flag indicating whether all field values must be distinct (as in ranking systems).
- `costExponent`: the exponent applied when computing the cost of casting votes in a field. This parameter enables quadratic or higher-order voting rules.
- `minValueSum`: the minimum total sum of all field values in a ballot.
- `maxValueSum`: the maximum total sum of all field values in a ballot (e.g., to enforce a budget of credits).

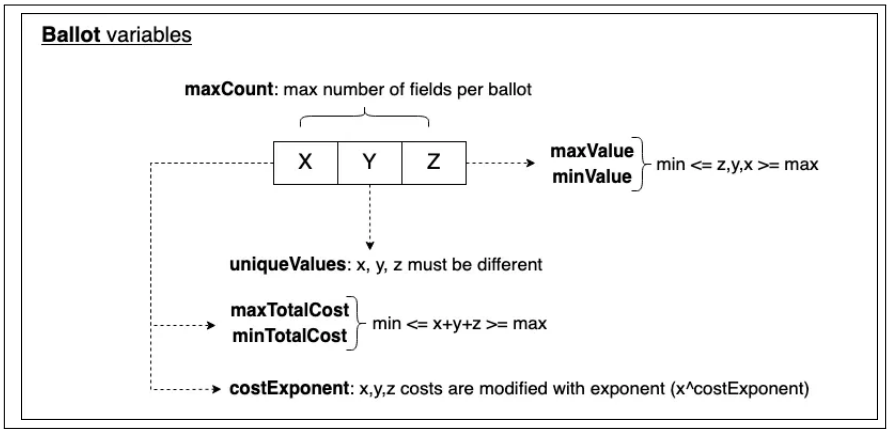

Fig. 9. Schematic representation of the parameters that define a ballot in DAVINCI (variable names are outdated).

General mechanism. A ballot is valid if and only if all the above constraints are satisfied. Invalid ballots are rejected at the circuit level and do not contribute to the tally. Valid ballots are aggregated into a results array, where each entry is the sum of all votes cast for the corresponding field across all voters. Voter weights, defined in the census tree (see Section 5.2.1), are taken into account when computing these sums. By default, all voters have the same weight, but the protocol also supports weighted voting, where different participants may contribute proportionally to their assigned voting power. In this case, `maxValueSum` reflects the maximum weight assigned to each voter, and the ballot and verifier circuits enforce that the correct weight is being used (see Sections 5.4.1 and 5.4.2 for further details).

Ballot modes. Ballot modes correspond to different voting systems, each adjusted with the variables above ballot configurations. For usability, the organizer does not need to set every parameter manually: instead, they simply select one of the predefined ballot modes (e.g., quadratic voting, ranking, approval), each corresponding to a fixed parameter configuration. Some representative modes are shown in Figure 10, which

illustrate the expressiveness of the ballot protocol. For example, in *approval voting*, each field is binary and voters can approve or reject multiple options simultaneously. In *ranking*, fields must form a permutation of the integers from 1 to N, enforcing uniqueness of values. In *quadratic voting*, voters allocate credits across options, and the cost is quadratic in the number of credits used, captured by setting `costExponent` $= 2$. Other systems such as single-choice or multiple-choice elections are special cases of the same framework. The figure also illustrates examples of voters' ballots, with the last column indicating whether each ballot is valid or invalid under the rules of the selected mode.

Name	Description	Example	numFields	minValue	maxValue	uniqueValues	costExponent	minValueSum	maxValueSum	Ballots example							
											1	2	3	4	5	totalValueSum	validVote
Approval voting	Multiple fields with only a 0-1 option	A referendum with 5 yes/no questions	5	0	1	FALSE	1	0	5	ballot 1	0	1	0	1	1	3	1
										ballot 2	1	1	1	1	1	5	1
										ballot 3	0	1	0	0	0	1	1
										total	**1**	**3**	**1**	**2**	**2**		**3**
											1	2	3	4	5	totalValueSum	validVote
Rating	Any field can be rated independently of the others	Satisfaction survey on 5 items, rated from 0 to 10 stars	5	0	10	FALSE	1	0	50	ballot 1	8	6	4	7	10	35	1
										ballot 2	5	4	6	12	4	31	0
										ballot 3	0	1	3	5	2	11	1
										total	**8**	**7**	**7**	**12**	**12**		**2**
											1	2	3	4	5	totalValueSum	validVote
Ranking	Number all options from 1 to N, using each number exactly once	Rank 5 destinations for the end-of-year school trip	5	1	5	TRUE	1	6	15	ballot 1	1	3	2	4	5	15	1
										ballot 2	1	5	3	4	5	18	0
										ballot 3	0	1	3	5	2	11	0
										total	**1**	**3**	**2**	**4**	**5**		**1**
											1	2	3	4	5	totalValueSum	validVote
Quadratic	Distribute M credits among N options (quadratically)	You have 12 credits to distribute among 5 options	5	0	12	FALSE	2	0	12	ballot 1	1	1	2	0	0	6	1
										ballot 2	3	0	0	0	2	13	0
										ballot 3	3	1	0	1	0	11	1
										total	**4**	**2**	**2**	**1**	**0**		**2**
											1	2	3	4	5	totalValueSum	validVote
Single choice	Select 1 among N options	Select a single winner from among 5 candidates	5	0	1	FALSE	1	1	1	ballot 1	1	0	0	0	0	1	1
										ballot 2	0	1	0	0	0	1	1
										ballot 3	0	1	1	0	0	2	0
										total	**1**	**1**	**0**	**0**	**0**		**2**
											1	2	3	4	5	totalValueSum	validVote
Multiple choice	Select M among N options	Select up to 3 finalists from 5 candidates	5	0	1	FALSE	1	0	3	ballot 1	1	0	1	1	0	3	1
										ballot 2	0	1	1	0	1	3	1
										ballot 3	1	2	3	0	0	6	0
										total	**1**	**1**	**2**	**1**	**1**		**2**

Fig. 10. Table showing various voting models using different ballot configurations. Each row corresponds to a voting system specified by a fixed configuration of the ballot parameters (`numFields`, `minValue`, `maxValue`, `uniqueValues`, `costExponent`, `minValueSum`, `maxValueSum`). The table also includes example ballots for each mode, illustrating how the constraints are enforced in practice. The last column indicates whether a ballot is valid (1) or invalid (0) under the rules of the selected mode.

7 The Vocdoni token

DAVINCI introduces the Vocdoni token (VOC) as a key element of its decentralized voting ecosystem, playing a crucial role in the protocol's sustainability. The token serves multiple utility functions that align the incentives of all participants (voting organizers, sequencers, and voters), ensuring the integrity, efficiency, and security of the system. In particular, the token has the following roles.

- *Collateral for sequencers.* Sequencers must stake VOC tokens as collateral to participate. This serves as a safeguard to ensure responsible participation. Misbehavior or failure to meet obligations can result in penalties, including partial or total loss of the stake.
- *Incentive mechanism.* Sequencers earn rewards in VOC tokens based on their contribution to processing valid votes and maintaining the network. Rewards are proportional to the number of valid votes successfully added to the shared state.
- *Payment for voting processes.* Organizers pay fees in VOC tokens to create and manage voting events. These fees depend on factors such as registry size, voting duration, and desired security level.
- *Governance.* Token holders can participate in the decentralized governance of the project, influencing protocol upgrades, ecosystem development, and other initiatives. This ensures that the project evolves in a transparent, community-driven manner.

7.1 Economics for organizers

Organizers cover the costs of voting processes in VOC tokens. The total cost combines four components:

$$\mathsf{totalCost} = \mathsf{baseCost} + \mathsf{capacityCost} + \mathsf{durationCost} + \mathsf{securityCost},$$

where

- baseCost is a fixed setup fee, independent of process duration or security level. It is calculated as

  $$\mathsf{baseCost} = \mathsf{fixedCost} + \mathsf{maxVotes} \cdot p,$$

  where fixedCost is a protocol-defined fee, maxVotes the maximum number of votes (votes or voters?), and p a linear factor. This portion is not reimbursable and always rewarded to sequencers.
- capacityCost accounts for limited sequencer capacity. That is, is models the cost of reserving space for voting events relative to the number of available sequencers, number of voting events running, and the maximum number of voters. Costs rise non-linearly as available capacity decreases as

  $$k_1 \cdot \left(\frac{\mathsf{totalVotingProcesses}}{\mathsf{totalSequencers} - \mathsf{usedSequencers} + \epsilon} \cdot \mathsf{maxVotes} \right)^a$$

  with k_1 a scaling factor, totalVotingProcesses the number of voting processes running, totalSequencers the number of registered sequencers, usedSequencers the number of sequencers handling other voting events, ϵ a small constant to avoid division by zero, and a an exponent controlling non-linearity.
- durationCost grows with the length of the voting period, scaled non-linearly with the formula

  $$k_2 \cdot \mathsf{processDuration}^b,$$

  where processDuration is measured in hours, k_2 is a scaling factor, and b controls non-linear growth. Shorter elections are cost-efficient, while longer ones become increasingly expensive.
- securityCost models the number of sequencers used, growing exponentially with diminishing returns:

  $$k_3 \cdot e^{c\left(\frac{\mathsf{numSequencers}}{\mathsf{totalSequencers}}\right)^d},$$

  where k_3 is a scaling factor, c controls the steepness, numSequencers is the number of sequencers needed in the process, totalSequencers the number of available sequencers, and d adjusts the non-linearity as the number of sequencers increases.
  Before totalSequencers was defined as the number of *registered* sequencers and here it means the number of *available* sequencers. Is it assumed to always be the same?

To avoid impractical scenarios, the following two constraints are enforced:

- If processDuration $>$ maxDuration, then totalCost $= \infty$.
- If numSequencers $>$ totalSequencers, then totalCost $= \infty$.

Reimbursements. Organizers initially reserve resources for all eligible voters, assuming maximum turnout. Since this rarely occurs, unused portions may be reimbursed. The reimbursement is defined as

$$\mathsf{reimbursement} = \mathsf{totalCost} - \mathsf{totalReward} - \mathsf{baseCost},$$

where totalReward is the actual amount distributed to sequencers based on their participation. This mechanism ensures organizers do not overpay for unused capacity, while sequencers are still compensated for committed resources.

7.2 Economics for sequencers

Sequencers must stake VOC in the sequencer registry smart contract to participate. Rewards are based on:

- The number of votes included in the shared state.
- The number of vote rewrites (either overwrites or re-encryptions). Rewrites enhance receipt-freeness and are incentivized, though limited by protocol constants.
- The ratio of processed to non-processed votes relative to the maximum allowed voters.

The reward function for the i-th sequencer is

$$\mathsf{sequencerReward}_i = R \cdot \left(\frac{\mathsf{votes}_i}{\mathsf{maxVotes}}\right) + W \cdot \left(\frac{\mathsf{voteRewrites}_i}{\mathsf{totalRewrites}}\right),$$

subject to the constraints

$$\frac{\mathsf{voteRewrites}_i}{\mathsf{votes}_i} \leq T, \quad \mathsf{totalReward} = R + W, \quad R > W,$$

where T is the maximum allowed ratio of rewrites to votes. Rewards prioritize new votes over rewrites, ensuring sequencers cannot maximize profits by simply re-encrypting existing ballots.

Penalties. Sequencers failing to provide required decryption shares or misbehaving face slashing penalties as

$$\mathsf{slashedAmount}_i = s \cdot \mathsf{stakedCollateral}_i,$$

where $0 \leq s \leq 1$ is a slashing coefficient. This ensures accountability and discourages free-riding.

7.3 Summary and remarks

The cost model combines four components: base, capacity, duration, and security. It ensures small processes are cost-efficient, large or resource-intensive processes incur higher costs, and impractical setups are excluded. Organizers aim to minimize costs, while sequencers seek to maximize rewards — a natural tension that can be modeled as a strategic game. Analyzing this equilibrium is left for future work. Move this to Section 7?

8 Analysis

The DAVINCI protocol was designed according to a set of guiding principles: cryptography is the sole source of truth; no single entity must be trusted; the system should be modular, open source, and resilient; and it should remain scalable, automated, and accessible to a wide range of users. Building on these principles, we now discuss the concrete security properties of the protocol, followed by implementation details and performance results. – This whole section is still [WIP].

→ check https://ethresear.ch/t/vocdoni-protocol-enabling-decentralized-voting-for-the-masses-with-zk-technology/21036.

8.1 Security discussion

Based on the above principles, the protocol provides a number of concrete security properties that ensure the integrity, confidentiality, and verifiability of voting events. In what follows we discuss how DAVINCI achieves these properties, with particular emphasis on receipt-freeness, privacy, unlinkability, and robustness against quantum threats.

Receipt-freeness. Receipt-freeness prevents voters from proving to others how they voted, thereby mitigating coercion and vote-buying. In DAVINCI this is achieved through re-encryption, ballot overwrites, and randomized state updates.

- *Ballot re-encryption*: since our encryption scheme supports re-randomization, that is, a ciphertext can be refreshed with new randomness without changing the underlying message, sequencers exploit this by re-encrypting the received ballots before committing them to the state Merkle tree, making it computationally infeasible to link a submitted ciphertext with the stored one (give details of what does *computationally infeasible* mean here).
- *Handling receipts*: because ballots are re-randomized, voters cannot generate a receipt by revealing the randomness r used in encryption, since the ciphertext on-chain no longer corresponds to their r. This blocks vote-selling and coercion.
- *Overwrites*: voters may cast a new ballot at any time, replacing their earlier submission. This ensures that even if coercion occurs, a voter can subsequently change their vote as many times as desired, preserving their ability to express their true choice. When an overwrite occurs, the sequencer substracts the previous ballot from the subtractive accumulator, adds the new ballot to the additive accumulator, and re-encrypts the updated ballot before storing it in the state tree.
- *Concealing overwrites*: to prevent observers from distinguishing overwrites from routine re-randomizations, sequencers periodically re-encrypt a random subset of stored ballots. This obfuscation ensures that overwrites remain indistinguishable from normal re-randomizations, strengthening receipt-freeness.

Privacy. Ballots remain secret even though encrypted ballots are publicly stored and processed. Ballot secrecy is preserved through ElGamal encryption, which allows votes to be aggregated without decryption. Encrypted ballots are stored in public repositories (e.g., Ethereum blobs). Because of DKG sequencers cannot decrypt individual ballots themselves. The protocol does not reveal the ballot but it does reveal if someone has voted or not.

Quantum resistance. Quantum computers threaten discrete-logarithm-based cryptography such as ECDSA and ElGamal. For this reason, DAVINCI is designed in a modular way that would allow to migrate to post-quantum primitives in the longer term. For example,use CRYSTALS-Dilithium, Falcon, or Rainbow, for signature schemes, Brakerski-Gentry-Vaikuntanathan (BGV) or Brakerski/Fan-Vercauteren (BFV) which are lattice-based homomorphic encryption schemes, and ZK-STARKs for post-quantum zero-knowledge proofs. Add citations to protocols.

Data availability. Ballots and state updates are stored in Ethereum blobs. Since these blobs may eventually be pruned from the blockchain, ensuring long-term data availability is an open problem. Possible solutions include decentralized storage networks or dedicated data-availability layers. This remains an active area of research.

End-to-end verifiability. Voters can check that their own ballot was included correctly, while anyone can audit the entire process to verify the final tally. Every voter can verify their ballot from casting to result computation (individual verifiability). Additionally, any third party can audit the election data to confirm results (universal verifiability) and verify that each vote comes from a uniquely registered voter (eligibility verifiability). Transparent cryptographic mechanisms make this possible.

8.2 Implementation

The system is modular, consisting of interchangeable components that can be rearranged or integrated with external systems via adaptable interfaces. This allows for redundancy, flexibility, and seamless integration with third-party applications, exemplified by our voting-as-a-service APIs. Vocdoni's voting platform (App) is open source, universally available and user-friendly. The interface is intuitive for all users, including those less familiar with technology, and accommodates voters that use assistive technologies like screen readers. By releasing our code openly, we invite anyone to audit and contribute, enhancing security and fostering community engagement. Transparency prevents security through obscurity and accelerates innovation. We minimize human intervention through smart contracts and cryptographic protocols, reducing costs and human error. Automation ensures consistent operation and frees resources for voter support and auditing. Add link to repositories and details of the software used.

Circuits.

- Circuit 1: circom/snarkJS, ~ 53.000 constraints.
- Circuit 2: gnark, ~ 3.1 million constraints.
- Circuit 3: gnark, $40.000 \times$ (number of votes) constraints.
- Circuit 4: gnark, ~ 16 million constraints.

8.3 Performance evaluation

Work in progress.

9 Conclusions

10 Future work

- Voter: they do circuit1 + circuit2 themselves (instead of the sequencer doing circuit2).
- Post-quantum.
- Support to xxx.

Acknowledgments

The authors would like to thank the following reviewers and contributors for their valuable feedback and support: all team members from the Vocdoni association, Jordi Baylina (from Iden3 and Polygon), Adrià Maçanet (Privacy Scaling Explorations, Ethereum Foundation), Arnaucube (0xPARC), Alex Kampa (AZKR), Roger Baig (Polytechnic University of Catalonia), Javier Herranz (Polytechnic University of Catalonia), and Jordi Puiggali (Secrets Vault).

References

1. Albrecht, M., Grassi, L., Rechberger, C., Roy, A., Tiessen, T.: Mimc: Efficient encryption and cryptographic hashing with minimal multiplicative complexity. In: Cheon, J.H., Takagi, T. (eds.) Advances in Cryptology – ASIACRYPT 2016. pp. 191–219. Springer Berlin Heidelberg, Berlin, Heidelberg (2016)
2. Baylina, J., Bellés, M.: Sparse Merkle trees (2019), https://docs.iden3.io/publications/pdfs/Merkle-Tree.pdf
3. Bellés-Muñoz, M., Whitehat, B., Baylina, J., Daza, V., Muñoz-Tapia, J.L.: Twisted edwards elliptic curves for zero-knowledge circuits. Mathematics **9**(23) (2021). https://doi.org/10.3390/math9233022, https://www.mdpi.com/2227-7390/9/23/3022
4. Bertoni, G., Daemen, J., Peeters, M., Van Assche, G.: The KECCAK SHA-3 submission (2011), https://keccak.team/files/Keccak-submission-3.pdf
5. Blake, I.F., Seroussi, G., Smart, N.P.: Elliptic curves in cryptography. Cambridge University Press, USA (1999)
6. Bowe, S., Chiesa, A., Green, M., Miers, I., Mishra, P., Wu, H.: ZEXE: enabling decentralized private computation. In: 2020 IEEE Symposium on Security and Privacy, SP 2020, San Francisco, CA, USA, May 18-21, 2020. pp. 947–964. IEEE (2020). https://doi.org/10.1109/SP40000.2020.00050, https://doi.org/10.1109/SP40000.2020.00050
7. Brown, D.R.L.: SEC 2: Recommended elliptic curve domain parameters. In: Standards for efficient cryptography 2 (SEC 2) (2010), https://www.secg.org/sec2-v2.pdf
8. El Housni, Y., Guillevic, A.: Optimized and secure pairing-friendly elliptic curves suitable for one layer proof composition. In: Krenn, S., Shulman, H., Vaudenay, S. (eds.) Cryptology and Network Security. pp. 259–279. Springer International Publishing, Cham (2020)
9. Grassi, L., Khovratovich, D., Rechberger, C., Roy, A., Schofnegger, M.: Poseidon: A new hash function for zero-knowledge proof systems. In: 30th USENIX Security Symposium (USENIX Security 21). pp. 519–535 (2021)
10. Groth, J.: On the size of pairing-based non-interactive arguments. In: Annual international conference on the theory and applications of cryptographic techniques. pp. 305–326. Springer (2016)
11. Housni, Y.E., Connor, M., Guillevic, A.: EIP-3026: BW6-761 curve operations [DRAFT], Ethereum Improvement Proposals, no. 3026, [online serial] (October 2020), https://eips.ethereum.org/EIPS/eip-3026

12. Jancar, J.: Standard curve database: BN254 (2020), https://neuromancer.sk/std/bn/bn254
13. Johnson, D., Menezes, A., Vanstone, S.: The elliptic curve digital signature algorithm (ECDSA). Int. J. Inf. Secur. **1**(1), 36–63 (August 2001). https://doi.org/10.1007/s102070100002, https://doi.org/10.1007/s102070100002
14. Sutikno, S., Surya, A., Effendi, R.: An implementation of elgamal elliptic curves cryptosystems. In: IEEE. APCCAS 1998. 1998 IEEE Asia-Pacific Conference on Circuits and Systems. Microelectronics and Integrating Systems. Proceedings (Cat. No.98EX242). pp. 483–486 (1998). https://doi.org/10.1109/APCCAS.1998.743829
15. Vlasov, A., hujw77: EIP-2539: BLS12-377 curve operations [DRAFT], Ethereum Improvement Proposals, no. 2539, [online serial] (February 2020), https://eips.ethereum.org/EIPS/eip-2539
16. WhiteHat, B., Bellés, M., Baylina, J.: ERC-2494: Baby Jubjub elliptic curve [DRAFT], Ethereum Improvement Proposals, no. 2494, [online serial] (January 2020), https://eips.ethereum.org/EIPS/eip-2494
17. Wood, G., et al.: Ethereum: A secure decentralised generalised transaction ledger. Ethereum project yellow paper **151**(2014), 1–32 (2014)